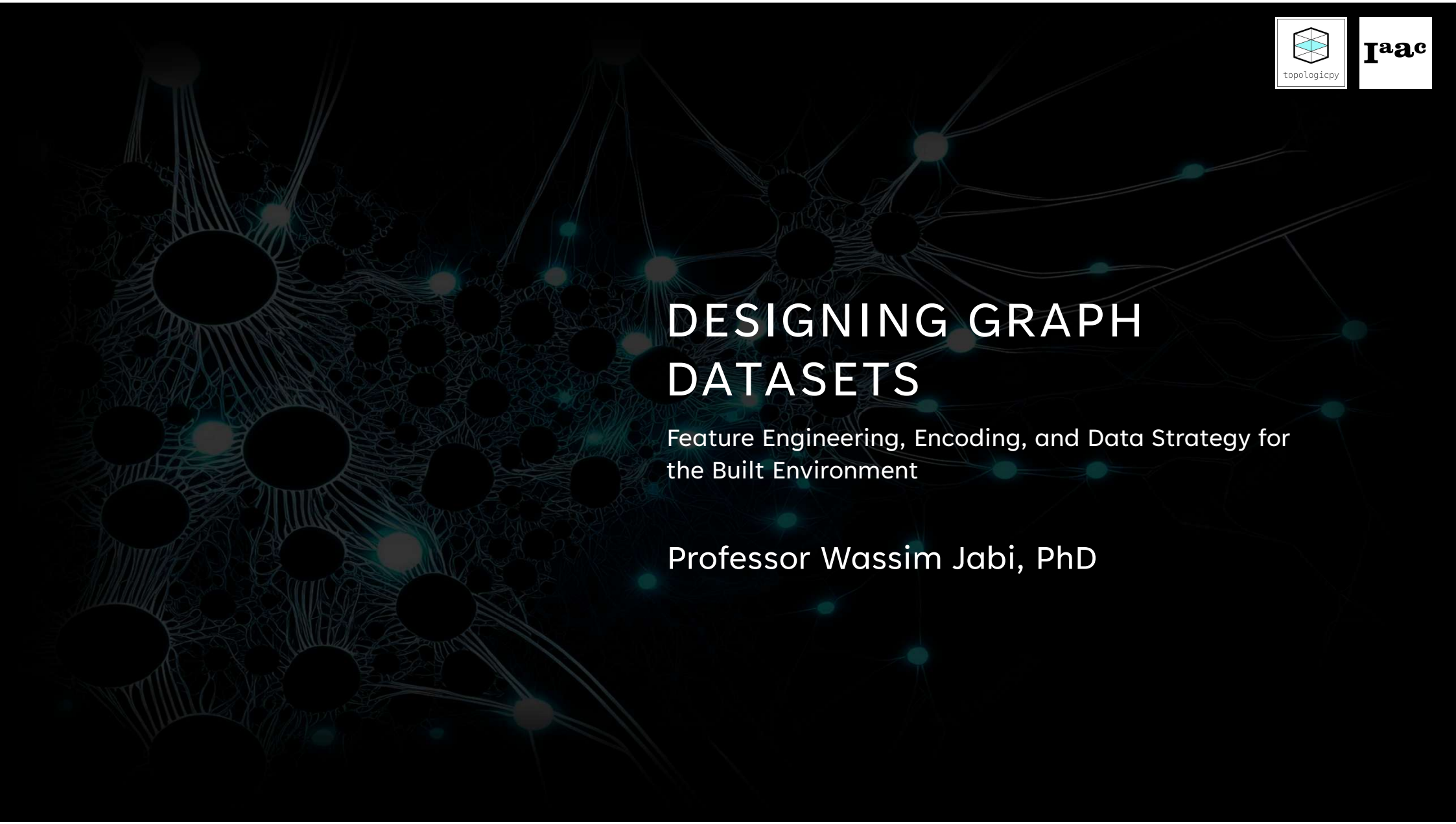

DESIGNING GRAPH DATASETS

Feature Engineering, Encoding, and Data Strategy for the Built Environment

Professor Wassim Jabi, PhD

OBJECTIVES

- How to design graph datasets that are meaningful, robust, and trainable
- Focus on datasets compatible with TopologicPy and PyTorch Geometric
- Emphasis on practical decisions: graph definition, feature engineering, encoding, targets, leakage, and evaluation

SIGNIFICANCE

- In graph machine learning, model performance is often determined more by dataset design than by model architecture
- In the built environment, graphs can encode:
  - rooms and adjacencies
  - building elements and relationships
  - circulation and visibility
  - energy systems and dependencies
  - urban networks and spatial interactions
- A graph dataset is a formal representation of a hypothesis about what matters in the built environment

CORE PRINCIPLES

- Start with the prediction or analysis task
- The task determines:
  - What the nodes are
  - What the edges are
  - What the features are
  - What the targets are
  - What counts as a sample
- Typical tasks:
  - Node-level: classify rooms, predict occupancy, estimate element performance
  - Edge-level: predict adjacency type, flow, connectivity, or relationship existence
  - Graph-level: predict building energy demand, walkability, typology, or performance rating
- Do not begin with: “We have IFC data, let us convert it into graphs.”
- Begin with: “What exactly are we trying to predict or explain?”

WHAT MAKES A GRAPH DATASET IN THE BUILT ENVIRONMENT?

A graph dataset requires five design decisions:

1. Target definition
   - What is the model trying to learn?
2. Graph definition (What is one training example?)
   - One building
   - One floor
   - One room network
   - One urban block
   - One visibility graph
3. Node definition (What entities become vertices?)
   - Rooms
   - Spaces
   - Walls
   - Windows
   - HVAC components
   - Parcels
   - Ontersections
4. Edge definition (What relations become edges?)
   - Adjacency
   - Containment
   - Visibility
   - Physical connection
   - Spatial overlap
   - Flow or access
   - Proximity
5. Feature definition
   - What measurable attributes are attached to nodes, edges, or the whole graph?

CONTINUOUS VS CATEGORICAL FEATURES

This distinction is essential.

- Independent features
  - The inputs given to the model. For example:
    - Room area
    - Window-to-wall ratio
    - Adjacency count
    - Material type
- Dependent variable / target
  - The output to be predicted. For example:
    - Energy consumption
    - Overheating risk
    - Room function class
    - Evacuation time class
    - Graph typology
- Rule
  - Inputs must be available at inference time
  - Targets must not leak into the features
- Simple test Ask: “Would I know this feature before I know the answer?” If no, it may be target leakage.

CONTINUOUS VS CATEGORICAL FEATURES

Use precise terminology.

- Continuous features
  - Numeric values with magnitude and ordering. For example:
    - Area
    - Height
    - Length
    - Temperature
    - Solar exposure
    - Degree centrality
  - Usually used directly, often after normalization or standardization
- Categorical features
  - Discrete classes or labels. For example:
    - Room type
    - Material class
    - Wall type
    - HVAC system category
    - Occupancy class
  - cannot usually be fed directly as arbitrary integers
- Do not confuse. room_type = 3 does not mean the room is “more” than room_type = 2

ONE-HOT ENCODING

One-Hot Encoding is a data preprocessing technique used to convert categorical variables into a numerical format that machine learning models can understand. It represents each category as a separate binary column, where a value of 1 indicates the presence of that category and 0 indicates its absence.

We use one hot Encoding because:

- Eliminating Ordinality: Many categorical variables have no inherent order (e.g., "Male" and "Female"). If we were to assign numerical values (e.g., Male = 0, Female = 1) the model might mistakenly interpret this as a ranking and lead to biased predictions. One Hot Encoding eliminates this risk by treating each category independently.
- Improving Model Performance: By providing a more detailed representation of categorical variables. One Hot Encoding can help to improve the performance of machine learning models. It allows models to capture complex relationships within the data that might be missed if categorical variables were treated as single entities.
- Compatibility with Algorithms: Many machine learning algorithms particularly based on linear regression and gradient descent which require numerical input. It ensures that categorical variables are converted into a suitable format.

ONE-HOT ENCODING

Fruit	Categorical value of fruit	Price
apple	1	5
mango	2	10
apple	1	15
orange	3	20

The output after applying one-hot encoding on the data is given as follows,

Fruit_apple	Fruit_mango	Fruit_orange	price
1	0	0	5
0	1	0	10
1	0	0	15
0	0	1	20

WHEN TO USE ONE-HOT ENCODING

- Use one-hot encoding when
  - categories are nominal
  - categories have no intrinsic order
  - the number of categories is manageable
  - interpretability matters
- Examples
  - room type: bedroom, kitchen, corridor, office
  - material category: concrete, timber, steel, glass
  - building use: residential, commercial, educational
- Why
  - one-hot encoding avoids introducing a false numeric ordering
  - it makes category membership explicit
- Example
  - room_type = kitchen
    becomes [0, 1, 0, 0, 0] rather than 2

WHEN NOT TO USE ONE-HOT ENCODING

One-hot encoding is not always the right choice.

- Avoid or reconsider when
  - the category space is very large
  - categories are sparse
  - categories are unstable across datasets
  - you have identifiers rather than meaningful categories
  - the dimensionality becomes excessive
- Examples
  - unique room IDs
  - unique building IDs
  - exact product codes
  - exact street names
  - contractor names
- Reason
  - one-hot encoding of identifiers usually teaches the model dataset-specific memorization
  - it increases dimensionality without adding generalizable meaning

WHEN NOT TO USE ONE-HOT ENCODING

One-hot encoding is not always the right choice.

- Avoid or reconsider when
  - The categories binned versions of continuous values, but still relate to each other.
  - The categories are cyclical.
- Examples
  - Window area is represented as: small, medium, large, and extra-large
    In this case, you might want to consider labelling them as 1,2,3,4 to indicate that “extra-large” is (on average) 4X greater than “small”

ORDINAL ENCODING VS ONE-HOT ENCODING

Some categories have meaningful order.

- Ordinal examples
  - Condition rating: poor, fair, good, excellent
  - Fire resistance class levels
  - Storey order
  - Accessibility level
  - Risk level
- In these cases
  - Ordinal encoding may be appropriate
  - One-hot may discard useful ordering information
- But be careful
  - Only use ordinal encoding when the order is real and meaningful
  - Never impose order where none exists

CYCLICAL ENCODING

Cyclical encoding is a feature transformation technique used for periodic variables that maps each value onto a unit circle using sine and cosine functions, thereby preserving their inherent circular continuity

Useful for representing cycles such as:

- Hours of the day
- Days of the week
- Months of the year
- Why use it?
  - If you represent days of the week (Sunday to Saturday) as (0 to 6). It will learn that Sunday (0) is very far from Saturday (6) when in fact they are adjacent.
- Solution: Use cyclical encoding to replace the feature with its *sin* and *cos* according to the formulas below:

$$x_{sin} = \sin\left(\frac{2\pi x}{T}\right), \quad x_{cos} = \cos\left(\frac{2\pi x}{T}\right)$$

Here, x is the encoded value (e.g., day index), and T is the period (e.g., 7 for days of the week).

This ensures that values at the boundaries (e.g., Saturday and Sunday) are **close in representation space**, unlike in one-hot or ordinal encoding.

The transformation produces two continuous features that capture the position on the cycle without introducing artificial discontinuities.

BINARY FEATURES

Binary features are often extremely useful.

- Examples
  - has_window
  - is_external
  - is_circulation_space
  - is_conditioned
  - is_load_bearing
  - has_direct_access_to_exit
- Strengths
  - Simple
  - Interpretable
  - low-dimensional
  - often robust
- Do not overproduce weak binary flags
  - too many poorly justified flags can create noisy, brittle datasets

FEATURE SCALING AND NORMALIZATION

Many models benefit from properly scaled continuous features.

- Common approaches
  - min-max scaling
  - z-score standardization
  - log transform for skewed features
  - area or length normalization
  - climate normalization for energy-related tasks
- Significance
  - prevents features with large magnitude from dominating
  - improves training stability
  - makes cross-building comparison more meaningful
- Important
  - compute scaling parameters using training data only then apply the same transform to validation and test data. Otherwise, this becomes a subtle form of leakage.

SPATIAL FEATURES: USE WITH CAUTION

Spatial coordinates are tempting, but risky.

- Potential features
  - x, y, z coordinates
  - centroid location
  - Orientation
  - height above ground
  - relative position within graph
- Problem
  - raw coordinates often encode project-specific layout rather than transferable knowledge
  - the model may memorize location rather than learn structure
- Better alternatives
  - relative coordinates
  - normalized position within building bounds
  - distance to entrance
  - distance to façade
  - depth from root or access node
  - graph-based position measures
- Rule: Use absolute coordinates only if the task genuinely depends on absolute position

STRUCTURAL, GEOMETRIC, AND SEMANTIC FEATURES

A strong built-environment graph dataset often combines three feature families:

- Geometric
  - Area, perimeter, volume, length, aspect ratio, compactness
- Topological / graph-theoretic
  - Degree
  - Centrality
  - Clustering coefficient
  - Shortest path depth
  - Visibility connectivity
  - Local neighbourhood descriptors
- Semantic
  - Room use
  - Material class
  - System type
  - Occupancy profile
  - Construction category
- Best practice
  - Combine feature families when they express different aspects of the phenomenon
  - Avoid including redundant versions of the same information

FEATURE REDUNDANCY AND COLLINEARITY

More features do not automatically mean better performance.

- Examples of redundant features
  - area and floor area in the same context
  - perimeter and boundary length when identical
  - degree and adjacency count
  - volume and area × height when one is derivable from the others
- Risks
  - inflated dimensionality
  - unstable importance estimates
  - harder interpretation
  - overfitting
- Good practice
  - inspect correlations
  - remove clearly redundant features
  - prefer compact and meaningful feature sets

FEATURE LEAKAGE

One the biggest challenges. Feature leakage occurs when input features contain information that should only be available after or because of the target.

- Examples
  - using annual energy consumption components to predict annual energy consumption
  - using a room label derived from the same annotation that defines the target
  - using post-simulation results as input to predict simulation outcomes
  - including graph metrics computed on the full network when the test node is not supposed to access future or hidden structure
- Result
  - unrealistic accuracy
  - poor real-world generalization
  - false confidence in the model

TRAIN, VALIDATION, AND TEST SPLITS IN GRAPH DATASETS

Splitting graph datasets is harder than in tabular ML.

- Possible split levels
  - graph-level split
  - building-level split
  - floor-level split
  - node-level split within graphs
  - temporal split
- Best practice for built environment
  - split by building or project where possible
  - do not let rooms from the same building leak into both train and test unless that is explicitly the deployment scenario
- Reason
  - buildings contain strong internal correlations
  - random node-level splitting can give overly optimistic results

INDUCTIVE VS TRANSDUCTIVE LEARNING

This distinction matters greatly in PyTorch Geometric.

- Inductive Learning
  - In classical rhetoric and later philosophy (notably Francis Bacon), “inductive reasoning” came to denote reasoning that leads from particular instances into general principles.
    - The model learns from some graphs and generalizes to unseen graphs
- Transductive Learning
  - The term was adapted in modern science (e.g., biology: signal transduction) and later formalised in machine learning by Vladimir Vapnik to describe inference that transfers or propagates information across a fixed set of instances, rather than generalising beyond them.
    - The model sees one large graph
    - Some nodes are train, some validation, some test

IMBALANCED DATASETS

Unequal class/category distribution. A very common issue in built-environment datasets.

- Examples
  - Many more “hospitals” than “libraries” in the dataset (graph-level)
  - Many more walls than doors or rare room functions (node-level)
  - Rare failure events or hazard categories
- Risks
  - Model predicts majority class well but fails where it matters
  - Misleading overall accuracy
- Strategies
  - Oversampling minority classes: Increase the representation of minority classes by duplicating or synthetically generating samples (e.g., via SMOTE) to balance the training distribution.
  - Undersampling majority classes: Reduce the number of majority class samples, either randomly or strategically, to achieve a more balanced class distribution during training.
  - Use class-balanced metrics: Evaluate model performance using metrics such as F1-score, precision–recall, or balanced accuracy that give equal importance to minority and majority classes rather than being dominated by the majority class.
  - Weighted losses: Modify the training objective by assigning higher penalties to errors on minority classes so the model learns to prioritise them despite their lower frequency.
  - Careful aggregation of rare categories where justified: Combine infrequent classes into a meaningful broader category when domain logic supports it, reducing sparsity while preserving semantic validity.***Do not report accuracy alone***

MISSING DATA

Built-environment datasets often contain incomplete or inconsistent metadata.

- Sources
  - incomplete IFC properties
  - inconsistent naming conventions
  - missing material assignments
  - undefined room functions
  - partial sensor coverage
- Options
  - dropping unreliable samples (brute force.. Not always recommended)
  - imputation (sophisticated substitution)
  - explicit “unknown” category
  - feature masking
  - creating robust reduced schemas

Missingness is itself sometimes informative. Do not hide it blindly.

GRAPH LABELS AND TARGET DESIGN

Targets must be clearly defined and stable

Target Types

- Regression
  - energy use intensity
  - heating load
  - travel time
- Classification
  - room category
  - risk class
  - building typology
- Ranking
  - priority or preference
- Multi-label
  - room has multiple functional tags

GRAPH LABELS AND TARGET DESIGN

- Good targets are
  - measurable
  - consistent
  - available at scale
  - ·relevant to the intended application
- Bad targets are
  - vague
  - inconsistently annotated
  - partly subjective without protocol

DESIGNING A FEATURE SCHEMA

A feature schema is a contract.

- It should specify:
  - feature name
  - entity type: node / edge / graph
  - data type: continuous / categorical / binary / ordinal
  - units
  - allowed range
  - encoding method
  - missing-value policy
  - source method
  - whether available at inference time
- Significance
  - reproducibility
  - consistent preprocessing
  - compatibility across graphs
  - easier debugging
  - safer model deployment

DO’S OF GRAPH DATASET DESIGN

- Define the learning task first
- Choose nodes and edges that match the phenomenon of interest
- Keep the ontology stable across samples
- Combine geometry, topology, and semantics thoughtfully
- Distinguish clearly between continuous, categorical, binary, and ordinal data
- Use one-hot encoding for nominal categories with manageable cardinality
- Normalize continuous features appropriately
- Split data by building or project when evaluating generalization
- Document the feature schema
- Check for leakage at every stage
- Inspect distributions, missingness, and imbalance before training
- Keep a simple baseline dataset before adding complexity

DON’TS OF GRAPH DATASET DESIGN

- Do not begin by extracting every possible feature
- Do not use arbitrary integer labels for nominal categories
- Do not one-hot encode meaningless identifiers
- Do not mix incompatible graph definitions in one dataset without justification
- Do not normalize using the full dataset before splitting
- Do not let rooms from the same building appear in both train and test unless intentional
- Do not include features unavailable at inference time
- Do not confuse correlation with causation
- Do not report accuracy alone on imbalanced problems
- Do not assume GNNs will fix poor dataset design

COMMON MISTAKES

- Using node IDs as features
- Treating room labels as both input and target
- Building graphs from inconsistent source conventions
- Ignoring units and scale
- Using absolute coordinates that encode project identity
- Using too many sparse categorical features
- Not separating semantic adjacency from geometric proximity
- Failing to distinguish observed from derived features
- Evaluating on near-duplicate buildings
- Overfitting to one project, one region, or one modelling standard

RIGOROUS VALIDATION QUESTIONS

Before training, ask:

- Is the graph definition aligned with the task?
- Are nodes comparable across all samples?
- Are edge semantics consistent?
- Are the features available at inference time?
- Have categorical variables been encoded correctly?
- Is there leakage in preprocessing, splitting, or feature construction?
- Are train and test graphs genuinely distinct?
- Are class distributions acceptable?
- Are missing values handled explicitly?
- Can the feature schema be reproduced by another researcher?

DEVELOPMENT STRATEGY

Build the dataset incrementally, not all at once.

- Stage 1: Minimal viable dataset
  - small number of clean features
  - clear target
  - stable graph construction
  - robust train/test split
- Stage 2: Add domain features
  - graph metrics
  - semantic classes
  - spatial descriptors
  - edge attributes
- Stage 3: Stress-test
  - leakage checks
  - ablation studies
  - feature importance
  - cross-project generalization
- Stage 4: Scale up
  - more buildings
  - more typologies
  - more climates
  - more heterogeneous graphs if justified

SUGGESTED ABLATION STUDIES

In graph machine learning, an ablation study is a systematic experimental procedure in which specific components of a model or input, such as node features, edge attributes, structural information, or layers of a GNN, are selectively removed or altered to quantify their individual contribution to overall model performance.

To understand what matters, test feature families separately. For example:

- Geometry only
- Topology only
- Semantics only
- Geometry + topology
- Topology + semantics
- Full model
- With and without edge features
- With and without one-hot semantics
- With and without positional features

Purpose

- Determine which information actually drives performance
- Identify redundant or misleading features
- Improve interpretability

FINALLY...

In the built environment, good datasets must integrate:

- Geometry
- Topology
- Semantics
- Task logic
- Evaluation discipline

- TopologicPy is powerful for constructing meaningful graph representations
- PyTorch Geometric is powerful for learning from them
- The bridge between the two is careful dataset design

The quality of a graph learning project depends first on how well the dataset expresses the structure of the built environment and the logic of the question being asked

THANK YOU

- Wassim Jabi
- wassim.jabi@gmail.com
- topologic.app